

Holographic Distance Formulas for Tree Token Graphs

Exact L^1 distances, Dynkin trees, and large-state examples

Farid Khafizov

CayleyPy project

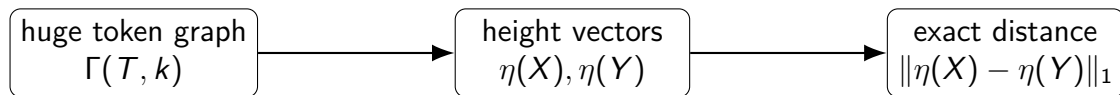
in collaboration with A.V.Chervov, F. Levkovich-Maslyuk, I.Litvak, A. Smolensky, et al

2026.05.10

df-v2l-tree-v4d-slides.tex

Core idea

A shortest-path problem in the token graph $\Gamma(T, k)$ can be replaced by a coordinate-wise computation on the base tree T .



The “holographic” data are cut counts across the edges of the tree.

Roadmap

- 1 Exact Tree Formula
- 2 Type- D_n Dynkin Trees
- 3 Type- E_n Dynkin Trees
- 4 Large-Scale Comparison
- 5 Takeaways

Roadmap

- 1 Exact Tree Formula
- 2 Type- D_n Dynkin Trees
- 3 Type- E_n Dynkin Trees
- 4 Large-Scale Comparison
- 5 Takeaways

Token graph $\Gamma(T, k)$

Definition

Let T be a finite tree with vertex set $[n]$. The k -token graph $\Gamma(T, k)$ has:

- vertices: all k -element subsets $X \subseteq [n]$;
- edges: one legal token move across one edge of T into an unoccupied vertex.

Equivalently, a configuration is a binary vector

$$X = (x_1, \dots, x_n) \in \{0, 1\}^n, \quad \sum_i x_i = k.$$

Subtree heights

Root the tree at r . Removing an edge e separates T into two components; let T_e be the component not containing r .

Height coordinate

$$\eta_e(X) = \sum_{i \in V(T_e)} x_i.$$

Height vector

$$\eta(X) = (\eta_e(X))_{e \in E(T)} \in \mathbb{Z}_{\geq 0}^{E(T)}.$$

Each coordinate counts how many tokens lie beyond one tree cut.

Reconstruction from heights

For a non-root vertex v , let e_v be the edge from v to its parent and let $C(v)$ be the set of children of v .

Linear inverse

$$x_v = \eta_{e_v}(X) - \sum_{w \in C(v)} \eta_{e_w}(X), \quad v \neq r.$$

At the root,

$$x_r = k - \sum_{w \in C(r)} \eta_{e_w}(X).$$

Therefore $X \mapsto \eta(X)$ is injective for every rooted tree.

Main theorem: holographic distance

Theorem (Tree distance formula)

For any two configurations $X, Y \in V(\Gamma(T, k))$,

$$d_{\Gamma(T, k)}(X, Y) = \sum_{e \in E(T)} |\eta_e(X) - \eta_e(Y)|.$$

This is an exact isometric embedding of the tree token graph into an integer L^1 lattice:

$$X \mapsto \eta(X), \quad \Gamma(T, k) \hookrightarrow \mathbb{Z}_{\geq 0}^{E(T)}.$$

Proof idea I: lower bound

A single token move crosses exactly one edge cut.

Single-move effect

If X' is obtained from X by one token move, then

$$\sum_{e \in E(T)} |\eta_e(X') - \eta_e(X)| = 1.$$

For any walk $X = Z_0, Z_1, \dots, Z_\ell = Y$,

$$\sum_e |\eta_e(X) - \eta_e(Y)| \leq \sum_{i=0}^{\ell-1} \sum_e |\eta_e(Z_{i+1}) - \eta_e(Z_i)| = \ell.$$

Taking the shortest walk gives the lower bound.

Proof idea II: upper bound

Define the edge imbalance

$$\delta_e = \eta_e(X) - \eta_e(Y).$$

- $\delta_e > 0$: tokens must cross e toward the root.
- $\delta_e < 0$: tokens must cross e away from the root.
- The required integral flow is forced by the tree cuts.

Because the support is acyclic, moves can be scheduled legally with no extra detours or collisions.

$$\text{number of moves} = \sum_{e \in E(T)} |\delta_e|.$$

Why the theorem is special to trees

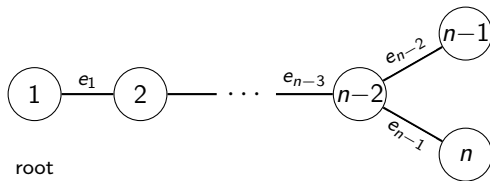
- In a tree, every edge is a cut edge.
- A token crossing an edge has no alternative route.
- Edge imbalances determine the unique transport demand.

For graphs with cycles, an edge is usually not a cut edge, so the same height coordinates do not force a unique transport schedule.

Roadmap

- 1 Exact Tree Formula
- 2 Type- D_n Dynkin Trees
- 3 Type- E_n Dynkin Trees
- 4 Large-Scale Comparison
- 5 Takeaways

The type- D_n tree



Edge set of the base Dynkin tree D_n :

$$E(D_n) = \{\{i, i+1\} : 1 \leq i \leq n-3\} \cup \{\{n-2, n-1\}, \{n-2, n\}\}.$$

D_n height vector

Root D_n at vertex 1.

Trunk heights

For $e_i = \{i, i+1\}$, $1 \leq i \leq n-3$,

$$\eta_{e_i}(X) = \sum_{j=i+1}^n x_j.$$

Branch heights

$$\eta_{\{n-2, n-1\}}(X) = x_{n-1}, \quad \eta_{\{n-2, n\}}(X) = x_n.$$

D_n holographic area

For configurations $X, Y \in \{0, 1\}^n$ with k tokens, define

$$\text{Area}_{D_n}(X, Y) = \sum_{i=1}^{n-3} \left| \sum_{j=i+1}^n (x_j - y_j) \right| + |x_{n-1} - y_{n-1}| + |x_n - y_n|.$$

Corollary

$$d_{\Gamma(D_n, k)}(X, Y) = \text{Area}_{D_n}(X, Y).$$

Diameter of $\Gamma(D_n, k)$

Theorem

For all $n \geq 4$ and $1 \leq k \leq n - 1$,

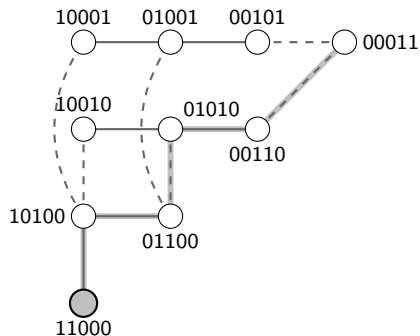
$$\text{diam } \Gamma(D_n, k) = k(n - k) - 1.$$

Equivalently,

$$\text{diam } \Gamma(D_n, k) = \text{diam } \Gamma(P_n, k) - 1.$$

The proof uses subtree edge capacities and the fact that the terminal fork removes exactly one unit from the path-token-graph diameter.

The full token graph $\Gamma(D_5, 2)$



Ten states, twelve edges; state $\{i, j\}$ drawn at grid position (i, j) . Solid edges: trunk moves (e_1, e_2); dashed edges: branch moves (e_3, e_4), which leave the grid. The shaded ribbon is a geodesic of length $5 = \text{diam } \Gamma(D_5, 2)$ from $A = 11000$ (filled) to $B = 00011$, matching $d(A, B) = \text{Area}_{D_5}(A, B) = 5$.

Explicit reconstruction for D_n

Write the height vector as

$$(h_1, \dots, h_{n-3}, a, b),$$

where $a = x_{n-1}$ and $b = x_n$ are the branch heights.

$$\begin{aligned}x_1 &= k - h_1, \\x_i &= h_{i-1} - h_i, \quad 2 \leq i \leq n-3, \\x_{n-2} &= h_{n-3} - a - b, \\x_{n-1} &= a, \quad x_n = b.\end{aligned}$$

This gives a concrete inverse to the D_n height map.

Worked example: $G_{100,25} = \Gamma(D_{100}, 25)$

$$|V(G_{100,25})| = \binom{100}{25} \approx 2.43 \cdot 10^{23},$$

$$|E(G_{100,25})| = 99 \binom{98}{24} \approx 4.55 \cdot 10^{24}.$$

For one random pair X, Y , the local edge contributions are:

$ \eta_e(X) - \eta_e(Y) $	0	1	2	3	4	5	6
# edges e	8	12	28	26	17	3	5

$$d_{\Gamma(D_{100},25)}(X, Y) = 259.$$

Why this matters computationally

For $G_{100,25}$:

- exhaustive BFS would face about $2.43 \cdot 10^{23}$ vertices;
- the average degree is 37.5;
- a complete traversal would inspect about $9.09 \cdot 10^{24}$ directed incidences.

The holographic formula computes the exact distance using only the 99 tree-edge coordinates:

$$d(X, Y) = \sum_{e \in E(D_{100})} |\eta_e(X) - \eta_e(Y)|.$$

Roadmap

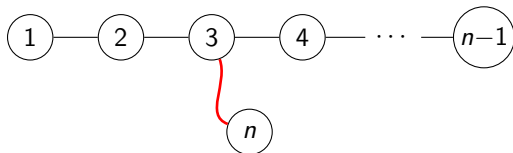
- 1 Exact Tree Formula
- 2 Type- D_n Dynkin Trees
- 3 Type- E_n Dynkin Trees
- 4 Large-Scale Comparison
- 5 Takeaways

The type- E_n tree

For the extended family E_n , use the trunk

$$1 - 2 - 3 - 4 - \cdots - (n - 1),$$

with an extra branch leaf n attached at vertex 3.



The branch is internal, not terminal.

E_n distance formula

The main tree theorem applies without modification:

$$d_{\Gamma(E_n, k)}(X, Y) = \sum_{e \in E(E_n)} |\eta_e(X) - \eta_e(Y)|.$$

What changes is the specialized coordinate form.

The branch leaf contributes to the first two trunk cuts, but not to the later trunk cuts. Thus the E_n suffix formula is less uniform than the D_n formula.

Example: $\Gamma(E_6, 2)$

For E_6 :

$$1 - 2 - 3 - 4 - 5, \quad 6 \text{ attached to } 3.$$

For $X = (x_1, \dots, x_6)$, ordered by edges (e_1, e_2, e_3, e_4, b) ,

$$\eta(X) = (x_2 + x_3 + x_4 + x_5 + x_6, x_3 + x_4 + x_5 + x_6, x_4 + x_5, x_5, x_6).$$

For $A = \{1, 2\}$ and $B = \{4, 5\}$,

$$\eta(A) = (1, 0, 0, 0, 0), \quad \eta(B) = (2, 2, 2, 1, 0), \quad d(A, B) = 6.$$

A geodesic in $\Gamma(E_6, 2)$

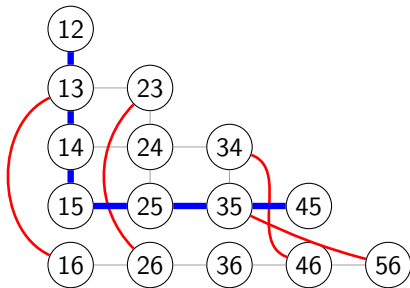
$$110000 \rightarrow 101000 \rightarrow 100100 \rightarrow 100010 \rightarrow 010010 \rightarrow 001010 \rightarrow 000110.$$

This is a geodesic of length 6 from $\{1, 2\}$ to $\{4, 5\}$.

Exhaustive BFS over the $\binom{6}{2} = 15$ states confirms

$$\text{diam } \Gamma(E_6, 2) = 6.$$

The full token graph $\Gamma(E_6, 2)$



A vertex label ij denotes the token configuration $\{i, j\}$.

The thick path is a geodesic of length 6 from 12 to 45.

The red curved edges are the moves induced by the branch edge $\{3, 6\}$ of E_6 .

Diameter comparison on six vertices

Tree T	Structure	$\text{diam } \Gamma(T, 2)$
P_6	path $1 - 2 - 3 - 4 - 5 - 6$	8
D_6	terminal fork	7
E_6	internal branch at 3	6

The central branch in E_6 shortens the worst-case distance more than the terminal fork in D_6 .

Open problem for E_n

Problem

Determine an exact formula for

$$\text{diam } \Gamma(E_n, k),$$

for the extended family E_n with $n \geq 6$ and $1 \leq k \leq n - 1$.

Equivalently: characterize which k -token configuration pairs maximize the E_n holographic area.

The edge-capacity upper bound need not be sharp when internal branching prevents simultaneous saturation.

Roadmap

- 1 Exact Tree Formula
- 2 Type- D_n Dynkin Trees
- 3 Type- E_n Dynkin Trees
- 4 Large-Scale Comparison**
- 5 Takeaways

Comparable scale: $5 \times 5 \times 5$ cube

A standard $5 \times 5 \times 5$ Professor's Cube has about

$$2.83 \times 10^{74}$$

reachable states.

A clean token-graph comparison is

$$\Gamma(D_{252}, 126), \quad |V| = \binom{252}{126} \approx 3.63 \times 10^{74}.$$

It has comparable state-space size, but the exact distance formula uses only

$$|E(D_{252})| = 251$$

height coordinates.

Token graph size for $\Gamma(D_{252}, 126)$

Let $G = \Gamma(D_{252}, 126)$.

$$|V(G)| = \binom{252}{126} \approx 3.63 \times 10^{74},$$

$$|E(G)| = 251 \binom{250}{125} \approx 2.289 \times 10^{76}.$$

The edge-to-vertex ratio is

$$\frac{|E(G)|}{|V(G)|} = 63,$$

so the average degree is

$$126.$$

Exact edge count

$$|E(\Gamma(D_{252}, 126))| = 22,893,300,098,974,613,611,622,017,384, \\ 487,282,592,294,844,338,217,541,432,733, \\ 522,920,094,075,250,752.$$

Even at this scale, distance is not found by searching the token graph; it is found by summing 251 cut contributions.

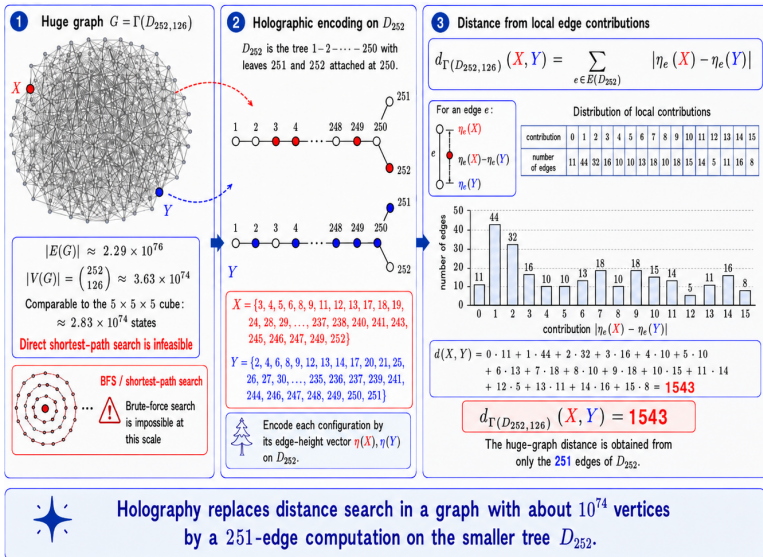
Example distance in $\Gamma(D_{252}, 126)$

For one random pair $X, Y \in V(\Gamma(D_{252}, 126))$, the local contributions are:

0	1	2	3	4	5	6	7
11	44	32	16	10	10	13	18
8	9	10	11	12	13	14	15
10	18	15	14	5	11	16	8

$$d(X, Y) = \sum_{e \in E(D_{252})} |\eta_e(X) - \eta_e(Y)| = 1543.$$

Holographic height profile in $\Gamma(D_{252}, 126)$



Puzzle-state comparison

Object	State count	Digits
$3 \times 3 \times 3$ Rubik's Cube	$\approx 4.33 \times 10^{19}$	20
$4 \times 4 \times 4$ Rubik's Revenge	$\approx 7.40 \times 10^{45}$	46
Megaminx	$\approx 1.01 \times 10^{68}$	69
$5 \times 5 \times 5$ Professor's Cube	$\approx 2.83 \times 10^{74}$	75
$\Gamma(D_{252}, 126)$	$\approx 3.63 \times 10^{74}$	75

Roadmap

- 1 Exact Tree Formula
- 2 Type- D_n Dynkin Trees
- 3 Type- E_n Dynkin Trees
- 4 Large-Scale Comparison
- 5 Takeaways

Takeaways

- ① For every finite tree T , the height-vector map embeds $\Gamma(T, k)$ isometrically into L^1 .
- ② The exact distance is the sum of absolute cut imbalances.
- ③ For D_n , this yields explicit suffix formulas, reconstruction, and

$$\text{diam } \Gamma(D_n, k) = k(n - k) - 1.$$

- ④ For E_n , the same theorem holds, but internal branching changes the closed forms and diameter behavior.
- ⑤ Enormous token-graph distances can be computed by small base-tree calculations.

References

-  R. Fabila-Monroy, D. Flores-Peñaloza, C. Huemer, F. Hurtado, J. Urrutia, and D. R. Wood, *Token graphs*, *Graphs and Combinatorics* 28 (2012), 365–380.
-  K. M. R. Audenaert, C. Godsil, G. Royle, and T. Rudolph, *Symmetric squares of graphs*, *Journal of Combinatorial Theory, Series B* 97 (2007), 74–90.
-  M. Mathey-Prévot and A. Valette, *Wasserstein distance and metric trees*, *L'Enseignement Mathématique* 69 (2023), no. 3–4, 315–333.

APPENDIX: CayleyPy Notes.

Path finding using the graph metric

Working only with finite groups G , there is a straightforward greedy best-first search algorithm for finding a path from an arbitrary vertex g to the vertex e using the word metric defined on $\Gamma(G, S)$. Namely, if the distance $d(g_1, g_2)$ is the length of the shortest path between g_1 and g_2 , the algorithm proceeds as follows:

- 1 Start at g .
- 2 From the vertices adjacent to g , choose g' such that $d(g', e)$ is minimal.
- 3 Replace g with g' and proceed until e is reached.

This works if d is known. If the function $d(g, e): G \rightarrow \mathbb{Z}_{\geq 0}$ is not known, we can (1) approximate it with a neural network or (2) compute / approximate it using holographic techniques.

Diffusion distance and beam search

For Cayley graphs, the training data consists of pairs $\{(g_i, d(g_i))\}$ obtained from a large number of random paths starting at e .

A neural network trained on this data approximates the average length of a path from a given vertex $g \in G$ to e . We call this function on the set of vertices the **diffusion distance** $DD(g)$.

We can pair the pathfinding algorithm with the diffusion distance heuristic, but this leads to inconsistent performance. A modification to the algorithm can be made as follows:

- ➊ Given an initial vertex g , choose vertices $g_1, \dots, g_N$ a known (minimal) distance away from g .
- ➋ Among all the unique vertices adjacent to the g_i , choose $g'_1, \dots, g'_N$ such that $DD(g'_i)$ is minimal.
- ➌ Replace g_i with g'_i and proceed until e is reached.

This is called **beam search**, and the parameter N is the “beam width”.

CayleyPy team results

Chervov et al. (2025) introduced a machine-learning method for pathfinding on extremely large Cayley graphs. The method combines a neural network approximation of diffusion distance with beam search and can solve the $3 \times 3 \times 3$, $4 \times 4 \times 4$, and $5 \times 5 \times 5$ Rubik's cubes, including cube sizes that classical methods cannot efficiently handle.

- $3 \times 3 \times 3$ **cube**: solved all scrambles from the DeepCubeA dataset with a 98.4% optimal-solution rate, compared with 60.3% for DeepCubeA and 69.6% for EfficientCube. Training required 4 h 40 min rather than 86 h 25 min, and the average solving time was 10.91 s rather than 287.78 s.
- $4 \times 4 \times 4$ **and** $5 \times 5 \times 5$ **cubes**: solved all corresponding scrambles from the 2023 Kaggle Santa Challenge dataset and produced shorter solutions than the best challenge submissions.

Bellman's equation: $d(g) = 1 + \min_{g' \sim g} d(g')$, $d(e) = 0$ can be used to approximate the true minimal-distance function. Starting with an arbitrary approximation $\tilde{d}$ satisfying $\tilde{d}(e) = 0$, one can improve it by minimizing the Bellman residual

$$1 + \min_{g' \sim g} \tilde{d}(g') - \tilde{d}(g).$$

One could initialize $\tilde{d} = DD$ and continue training the neural network using

$$\mathcal{B}(\theta) = \sum_i \left(1 + \min_{g' \sim g_i} DD_\theta(g') - DD_\theta(g_i) \right)^2.$$

This principle is also central to reinforcement-learning algorithms such as Q-learning and its neural-network variant, DQN.

Graph embeddings

Being able to accurately model distance on a graph seems to have an additional hidden requirement: an embedding into Euclidean space.

- Given a graph Γ , we want an embedding $V_\Gamma \hookrightarrow \mathbb{R}^N$ such that N is minimal, together with a function f defined on the image.
- Perform beam search using f as a heuristic.
- For $G \hookrightarrow S_n$, we use the representation of S_n in $\mathbb{R}^n$ by permutation of the coordinates. The orbit of a single vector $[0, 1, \dots, n]$ gives an embedding of the vertices

$$V_{\Gamma(G,S)} \hookrightarrow \mathbb{R}^n.$$

The function f is a neural-network approximation of DD or $d(-, e)$.

- In general, the criteria for a good embedding are a small value of N and a successful search with a small beam width.
- Ideally, the embedding would be an isometry, since this makes the search easy using Euclidean distance. Finding the diameter is also easy.